

P2418R1

Add support for `std::generator`-like types to `std::format`

Published Proposal, 2021-09-11

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 **Proposal**
- 2 **Changes since R0**
- 3 **LEWG polls**
- 4 **Impact on existing code**
- 5 **Implementation experience**
- 6 **Wording**
- 7 **Acknowledgements**

References

Informative References

§ 1. Proposal

[P2286] raised an issue of formatting `std::generator` from [P2168] and similar views with C++20 `std::format`. The issue is illustrated in the following example:

```
auto ints_coro(int n) -> std::generator<int> {
    for (int i = 0; i < n; ++i) {
        co_yield i;
    }
}
std::format("{} ", ints_coro(10)); // error
```

Unfortunately we cannot make `std::generator` formattable because it is neither const-iterable nor copyable and `std::format` takes arguments by `const&`. This hasn't been a problem in C++20 because range adapters which can also be not const-iterable are usually copyable. However, it will likely become a problem in the future once coroutines are more widely adopted.

This paper proposes solving the issue by making `std::format` and other formatting functions take arguments by forwarding references.

Other benefits of using forwarding references:

- Formatting of non-const-iterable views can be more efficient by avoiding a copy.
- It becomes possible to detect common lifetime errors, for example:

```
auto joined = std::format_join(std::vector{10, 20, 30, 40, 50, 60}, ":")
std::format("{:02x}", joined); // UB but can be made ill-formed with th
```

§ 2. Changes since R0

- Added LEWG poll results.
- Improved the wording.
- Added the Acknowledgements section.

§ 3. LEWG polls

Poll: Send P2418R0 (Adding support for `std::generator` to `std::format`) to LWG for C++23 and as a DR for C++20, treated as an urgent matter.

SF	F	N	A	SA
11	5	1	0	0

§ 4. Impact on existing code

This change will break formatting of bit fields:

```
struct S {
    int bit: 1;
};

auto s = S();
std::format("{} ", s.bit); // will become ill-formed
```

Supporting bit fields was one of the reasons `std::format` passed arguments by `const&` in the first place. However, there are simple workarounds for this:

```
std::format("{} ", +s.bit); // use + or cast to int
```

§ 5. Implementation experience

The proposal has been implemented in the `{fmt}` library. Arguments have been passed by forwarding references since `{fmt}` 6.0 released about two years ago and non-`const&` argument support in `formatter` specializations was added recently.

§ 6. Wording

All wording is relative to the C++ working draft [\[N4892\]](#).

Update the value of the feature-testing macro `__cpp_lib_format` to the date of adoption in [\[version.syn\]](#):

Change in [\[format.syn\]](#):

```
namespace std {
    // [format.functions], formatting functions
    template<class... Args>
        string format(format-string<Args...> fmt, const Args&Args&&... args);
    template<class... Args>
        wstring format(wformat-string<Args...> fmt, const Args&Args&&... args);
    template<class... Args>
        string format(const locale& loc, format-string<Args...> fmt,
            const Args&Args&&... args);
    template<class... Args>
        wstring format(const locale& loc, wformat-string<Args...> fmt,
            const Args&Args&&... args);

    ...

    template<class Out, class... Args>
        Out format_to(Out out, format-string<Args...> fmt, const Args&Args&&... args);
    template<class Out, class... Args>
        Out format_to(Out out, wformat-string<Args...> fmt, const Args&Args&&... args);
    template<class Out, class... Args>
        Out format_to(Out out, const locale& loc, format-string<Args...> fmt,
            const Args&Args&&... args);
    template<class Out, class... Args>
        Out format_to(Out out, const locale& loc, wformat-string<Args...> fmt,
            const Args&Args&&... args);

    ...

    template<class Out, class... Args>
        format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
            format-string<Args...> fmt,
            const Args&Args&&... args);
    template<class Out, class... Args>
        format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
            wformat-string<Args...> fmt,
            const Args&Args&&... args);
    template<class Out, class... Args>
        format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
            const locale& loc,
            format-string<Args...> fmt,
            const Args&Args&&... args);
```

```

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                        const locale& loc,
                                        wformat_string<Args...> fmt,
                                        const Args&Args&&... args);

template<class... Args>
    size_t formatted_size(format_string<Args...> fmt, const Args&Args&&... args);
template<class... Args>
    size_t formatted_size(wformat_string<Args...> fmt, const Args&Args&&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, format_string<Args...> fmt,
                        const Args&Args&&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, wformat_string<Args...> fmt,
                        const Args&Args&&... args);

...

template<class Context = format_context, class... Args>
    format_arg_store<Context, Args...>
        make_format_args(const Args&Args&&... fmt_args);
template<class... Args>
    format_arg_store<wformat_context, Args...>
        make_wformat_args(const Args&Args&&... args);

...
}

```

Change in [\[format.functions\]](#):

```

template<class... Args>
    string format(format_string<Args...> fmt, const Args&Args&&... args);

```

...

```

template<class... Args>
    wstring format(wformat_string<Args...> fmt, const Args&Args&&... args);

```

...

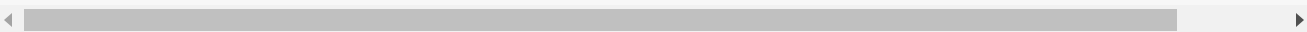
```
template<class... Args>
    string format(const locale& loc, format-string<Args...> fmt,
        const Args&Args&&... args);
```

...

```
template<class... Args>
    wstring format(const locale& loc, wformat-string<Args...> fmt,
        const Args&Args&&... args);
```

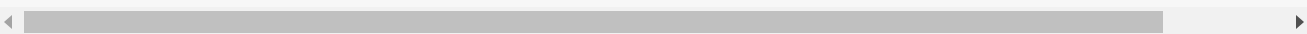
...

```
template<class Out, class... Args>
    Out format_to(Out out, format-string<Args...> fmt, const Args&Args&&... a
```



...

```
template<class Out, class... Args>
    Out format_to(Out out, wformat-string<Args...> fmt, const Args&Args&&...
```



...

```
template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, format-string<Args...> fmt,
        const Args&Args&&... args);
```

...

```
template<class Out, class... Args>
    Out format_to(Out out, const locale& loc, wformat-string<Args...> fmt,
        const Args&Args&&... args);
```

...

```
template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         format_string<Args...> fmt,
                                         const Args&Args&&... args);

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         wformat_string<Args...> fmt,
                                         const Args&Args&&... args);

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         const locale& loc,
                                         format_string<Args...> fmt,
                                         const Args&Args&&... args);

template<class Out, class... Args>
    format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
                                         const locale& loc,
                                         wformat_string<Args...> fmt,
                                         const Args&Args&&... args);
```

...

Preconditions: `Out` models `output_iterator<const charT&>`, and
`formatter<Ti, remove_cvref_t<Ti>, charT>` meets the *Formatter* requirements
([`formatter.requirements`]) for each `Ti` in `Args`.

...

```
template<class... Args>
    size_t formatted_size(format_string<Args...> fmt, const Args&Args&&... args);
template<class... Args>
    size_t formatted_size(wformat_string<Args...> fmt, const Args&Args&&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, format_string<Args...> fmt,
                         const Args&Args&&... args);
template<class... Args>
    size_t formatted_size(const locale& loc, wformat_string<Args...> fmt,
                         const Args&Args&&... args);
```

...

Preconditions: `formatter<F, remove_cvref_t<Ti>, charT>` meets the *Formatter* requirements ([`formatter.requirements`]) for each `Ti` in `Args`.

...

Change in [`formatter.requirements`]:

A type `F` meets the *BasicFormatter* requirements if:

- it meets the
 - *Cpp17DefaultConstructible* (Table 27),
 - *Cpp17CopyConstructible* (Table 29),
 - *Cpp17CopyAssignable* (Table 31), and
 - *Cpp17Destructible* (Table 32)
- requirements,
- it is swappable ([`swappable.requirements`]) for lvalues, and
- the expressions shown in ~~Table 67~~ `Table [tab:basic.formatter]` are valid and have the indicated semantics.

A type `F` meets the *Formatter* requirements if it meets the *BasicFormatter* requirements and the expressions shown in Table 67 are valid and have the indicated semantics. All types that have `formatter` specializations satisfy the *Formatter* requirements unless specified otherwise.

...

Given character type `charT`, output iterator type `Out`, and formatting argument type `T`, in ~~Table~~ `Tables [tab:basic.formatter]` and 67:

- `f` is a value of type `F`,
- `u` is an lvalue of type `T`,
- `t` is a value of a type convertible to (possibly const) `T`,
- `PC` is `basic_format_parse_context<charT>`,
- `FC` is `basic_format_context<Out, charT>`,
- `pc` is an lvalue of type `PC`, and
- `fc` is an lvalue of type `FC`.

`pc.begin()` points to the beginning of the *format-spec* ([format.string]) of the replacement field being formatted in the format string. If *format-spec* is empty then either `pc.begin() == pc.end()` or `*pc.begin() == '}'`.

Table 67: *BasicFormatter* requirements [tab:basic.formatter]

Expression	Return type	Requirement
<code>f.parse(pc)</code>	<code>PC::iterator</code>	Parses <i>format-spec</i> (20.20.2) for type T in the range <code>[pc.begin(), pc.end())</code> until the first unmatched character. Throws <code>format_error</code> unless the whole range is parsed or the unmatched character is <code>}</code> . [Note 1: This allows formatters to emit meaningful error messages. — end note] Stores the parsed format specifiers in <code>*this</code> and returns an iterator past the end of the parsed range.
<code>f.format(u, fc)</code>	<code>FC::iterator</code>	Formats <code>u</code> according to the specifiers stored in <code>*this</code> , writes the output to <code>fc.out()</code> and returns an iterator past the end of the output range. The output shall only depend on <code>u</code> , <code>fc.locale()</code> , <code>fc.arg(n)</code> for any value <code>n</code> of type <code>size_t</code> , and the range <code>[pc.begin(), pc.end())</code> from the last call to <code>f.parse(pc)</code> .

Table 67: *Formatter* requirements [tab:formatter]

Expression	Return type	Requirement
<code>f.parse(pc)</code>	<code>PC::iterator</code>	Parses <i>format-spec</i> (20.20.2) for type T in the range <code>[pc.begin(), pc.end())</code> until the first unmatched character. Throws <code>format_error</code> unless the whole range is parsed or the unmatched character is <code>}</code> . [Note 1: This allows formatters to emit meaningful error messages. — end note] Stores the parsed format specifiers in <code>*this</code> and returns an iterator past the end of the parsed range.
<code>f.format(t, fc)</code>	<code>FC::iterator</code>	Formats <code>t</code> according to the specifiers stored in <code>*this</code> , writes the output to <code>fc.out()</code> and returns an iterator past the end of the output range. The output shall only depend on <code>t</code> , <code>fc.locale()</code> , <code>fc.arg(n)</code> for any value <code>n</code> of type <code>size_t</code> , and the range

Expression	Return type	Requirement
		[<code>pc.begin()</code> , <code>pc.end()</code>) from the last call to <code>f.parse(pc)</code> .
<code>f.format(u, fc)</code>	<code>FC::iterator</code>	As above, but does not modify <code>u</code> .

Change in `[format.arg]`:

```
namespace std {
    template
    class basic_format_arg {
    private:
        ...
        template<class T> explicit basic_format_arg(const T& v) noexcept; /
        ...
    }
}
```

...

```
template<class T> explicit basic_format_arg(const T& v) noexcept;
```

Constraints: The template specialization

```
typename Context::template formatter_type<remove_cvref_t<T>>
```

meets the *Formatter* requirements ([`formatter.requirements`]). The extent to which an implementation determines that the specialization meets the *Formatter* requirements is unspecified, except that as a minimum the expression

```
typename Context::template formatter_type<remove_cvref_t<T>>()
    .format(declval<const T&>(), declval<Context&>())
```

shall be well-formed when treated as an unevaluated operand.

...

The class `handle` allows formatting an object of a user-defined type.

```
namespace std {
    template<class Context>
    class basic_format_arg<Context>::handle {
        const void* ptr_; // exposi
        void (*format_)(basic_format_parse_context<char_type>&,
                        Context&, const void*); // exposi

        template<class T> explicit handle(const T&& val) noexcept; // exposi
        ...
    };
}
```

```
template<class T> explicit handle(const T&& val) noexcept;
```

Let

- `TD` be `remove_cvref_t<T>`,
- `const-formattable` be true if `typename Context::template formatter_type<TD>().format(declval<const TD&>(), declval<Context&>())` is well-formed, otherwise false,
- `TQ` be `const TD` if `const-formattable` is true and `TD` otherwise.

Mandates: `const-formattable` `||` `!is_const_v<remove_reference_t<T>>` is true.

Effects: Initializes `ptr_` with `addressof(val)` and `format_` with

```
[](basic_format_parse_context<char_type>& parse_ctx,
    Context& format_ctx, const void* ptr) {
    typename Context::template formatter_type<TD> f;
    parse_ctx.advance_to(f.parse(parse_ctx));
    format_ctx.advance_to(f.format(
        *static_cast<const T*>(ptr), const_cast<TQ*>(static_cast<const TD*>(ptr))
    ));
}
```

Change in [\[format.arg.store\]](#):

```
template<class Context = format_context, class... Args>
    format_arg_store<Context, Args...> make_format_args(const Args&Args&&...

...

template<class... Args>
    format_arg_store<wformat_context, Args...> make_wformat_args(const Args&Args&&...
```

Add to [\[diff.cpp20.utilities\]](#):

Affected subclause: 20.20

Change: Signature changes: [format](#), [format_to](#), [format_to_n](#), [formatted_size](#).

Rationale: Enable formatting of views that are neither const-iterable nor copyable.

Effect on original feature: Valid C++20 code that passed bit fields to formatting functions may become ill-formed. For example:

```
struct tiny {
    int bit: 1;
};

auto t = tiny();
std::format("{} ", t.bit); // ill-formed,
                           // previously returned "0"
```

§ 7. Acknowledgements

Thanks Barry Revzin for bringing up the issue of formatting `std::generator` in [\[P2286\]](#). Thanks Tim Song and Tomasz Kamiński for wording improvement suggestions.

§ References

§ Informative References

[N4892]

Thomas Köppe. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4892). 18 June 2021. URL: <https://wg21.link/n4892>

[P2168]

Lewis Baker; Corentin Jabot. [std::generator: Synchronous Coroutine Generator for Ranges](https://wg21.link/p2168). URL: <https://wg21.link/p2168>

[P2286]

Barry Revzin. [Formatting Ranges](https://wg21.link/p2286). URL: <https://wg21.link/p2286>